

AgriVista Field

DOSSIER TECHNIQUE FINAL

Socle obligatoire + fonctionnalités optionnelles

Élément	Information
Formation	Master BIHAR
Technologie	Flutter 3.44.8 - Dart 3.12.2
Projet	Application mobile de gestion d'interventions terrain
Auteur	ARGAILLOT Dorian
Session	2025-2026

Finalité du document

Décrire l'état final réellement livré d'AgriVista Field : fonctionnalités, architecture, gestion d'état, source JSON, persistance locale, responsive, tests, limites et conformité au sujet.

Version finale après validation des quatre extensions optionnelles

Table des matières

1. Objet du document.....	4
2. Objectifs couverts.....	4
3. Fonctionnalités réalisées.....	4
3.1 Socle obligatoire.....	4
3.2 Extensions optionnelles.....	4
4. Stack et versions.....	5
5. Architecture.....	5
5.1 Principes.....	5
5.2 Arborescence réelle.....	6
5.3 Composition root.....	6
6. Gestion d'état avec Riverpod.....	7
7. Source JSON et Dio.....	7
7.1 Endpoint.....	7
7.2 Configuration réseau.....	7
7.3 Validation des données.....	7
7.4 Stratégie distant/local.....	7
8. Persistance locale.....	9
8.1 Statuts.....	9
8.2 Notes.....	9
8.3 Photos.....	9
8.4 Préférence de thème.....	9
9. Interface et responsive.....	9
9.1 Thèmes.....	9
9.2 Téléphone.....	9
9.3 Tablette.....	10
10. Gestion des erreurs.....	11
11. Tests et validation.....	11
11.1 Suite automatisée.....	11
11.2 Validation Android.....	11
11.3 Validation tablette.....	11
11.4 Validation iOS.....	12
12. Difficultés rencontrées et solutions.....	13
13. Limites connues.....	13
14. Améliorations futures.....	13
15. Commandes de reproduction.....	14
16. Conclusion.....	14
Annexe A — Matrice de conformité au sujet.....	15
A.1 Socle obligatoire et exigences de remise.....	15
A.2 Extensions optionnelles.....	15

Annexe B — Livraison et versionnement.....	17
Livrables de remise :.....	17

Lecture rapide

Le dossier distingue clairement le socle obligatoire des quatre extensions optionnelles : Dashboard, Refresh + tri, Note + photo locale, Thème + tablette.

1. Objet du document

Ce dossier décrit l'état final de l'application Flutter AgriVista Field après réalisation du socle obligatoire et des quatre extensions optionnelles. Il présente les choix techniques réellement présents dans le repository, leur validation et leurs limites.

AgriVista Field est destinée aux techniciens chargés de l'installation et de la maintenance de stations de capteurs agricoles et viticoles. L'application consulte les interventions publiées dans un JSON distant, permet de préparer et suivre leur exécution, puis conserve localement les informations terrain non synchronisées.

2. Objectifs couverts

- Charger et valider une source JSON distante.
- Afficher la liste et le détail des interventions.
- Rechercher par station, domaine ou description et filtrer par statut/priorité.
- Trier par date prévue, priorité ou statut.
- Faire progresser une intervention de planifiée vers en_cours, puis terminée.
- Conserver localement les statuts modifiés, les notes, les photos et la préférence de thème.
- Présenter un dashboard synthétique et actualiser les données par pull-to-refresh.
- Choisir un thème Système, Clair ou Sombre.
- Adapter la navigation et le détail aux écrans de tablette.
- Représenter explicitement les états loading, empty et error.

3. Fonctionnalités réalisées

3.1 Socle obligatoire

- Consommation HTTP du JSON officiel avec Dio et désérialisation typée avec Freezed / json_serializable.
- Mapping séparé des DTO vers les entités métier.
- Liste, recherche textuelle, filtres sur les trois statuts et les trois priorités.
- Détail complet incluant date prévue, coordonnées, description, statut et historique.
- Transition de statut séquentielle et irréversible.
- Surcharge locale du statut distant avec Hive.
- Profil construit depuis le technicien reçu, navigation Material 3 et états asynchrones explicites.

3.2 Extensions optionnelles

Extension	Réalisation
Dashboard	Total et compteurs par statut/priorité ; clic sur une métrique pour ouvrir la liste filtrée.
Actualisation + tri	Pull-to-refresh non destructif ; tri date/priorité/statut et inversion de l'ordre.
Compte rendu terrain	Note locale et photo locale persistées par intervention, modifiables depuis le détail.
Thème + tablette	Système/Clair/Sombre persistés ; NavigationRail et master/détail à partir de 720 px.

4. Stack et versions

Les versions ci-dessous correspondent à l'environnement de validation et aux versions résolues dans pubspec.lock.

Élément	Version	Usage
Flutter	3.44.8	Framework multiplateforme, widgets et Material 3
Dart	3.12.2	Langage et Domain pur
flutter_riverpod	3.4.2	Gestion d'état et injection
dio	5.11.0	Client HTTP du JSON distant
freezed / annotation	3.2.5 / 3.1.0	DTO immuables et génération
json_serializable / annotation	6.14.1 / 4.12.0	Désérialisation JSON
hive_flutter / hive	1.1.0 / 2.2.3	Persistance locale clé/valeur
image_picker	1.2.3	Sélection d'une photo dans la galerie
path_provider	2.1.6	Accès au stockage permanent de l'application
build_runner	2.15.1	Génération de code
flutter_lints	6.0.0	Analyse statique

Choix assumé

Aucun package supplémentaire n'est nécessaire pour la navigation ou le responsive : les composants Material 3 natifs de Flutter suffisent.

5. Architecture

5.1 Principes

L'application suit une Clean Architecture orientée par fonctionnalités :

Presentation → Domain ← Data

- Domain : entités, enums, contrats de repositories et cas d'usage en Dart pur. Aucun import Flutter, Riverpod, Dio ou Hive.
- Data : HTTP, JSON, Hive, fichiers locaux, DTO, mapping et implémentations de repositories.
- Presentation : widgets, pages et providers Riverpod.
- Core : erreurs, client réseau, thèmes, breakpoint responsive et widgets génériques.
- App : MaterialApp, shell de navigation et composition root.

5.2 Arborescence réelle

```

lib/
├── app/
│   ├── app.dart
│   ├── app_constants.dart
│   ├── app_shell.dart
│   └── bootstrap.dart
├── core/
│   ├── errors/
│   ├── network/
│   ├── theme/
│   ├── utils/
│   └── widgets/
├── features/
│   ├── dashboard/presentation/
│   ├── interventions/
│   │   ├── data/{datasources,mappers,models,repositories}/
│   │   ├── domain/{entities,repositories,usecases}/
│   │   └── presentation/{pages,providers,widgets}/
│   └── profile/presentation/{pages,widgets}/
└── main.dart

```

5.3 Composition root

bootstrap.dart initialise l'infrastructure avant runApp :

1. Initialisation Flutter et Hive.
2. Ouverture des quatre boxes locales.
3. Construction du client Dio et des data sources.
4. Construction des repositories concrets.
5. Injection des repositories, du sélecteur photo et de la préférence de thème dans ProviderScope.
6. Lancement de AgriVistaApp.

Conséquence

Les widgets ne construisent jamais directement Dio, Hive ou ImagePicker. Les dépendances techniques restent injectées et testables.

6. Gestion d'état avec Riverpod

Provider	Responsabilité
interventionsProvider	Chargement global, actualisation et mise à jour de la liste après changement de statut.
interventionFiltersProvider	Recherche, filtres, critère et direction de tri.
dashboardSummaryProvider	Projection des interventions en compteurs de synthèse.
compteRenduProvider(id)	Lecture et mutation de la note/photo d'une intervention.
themeModeProvider	Restauration et modification du ThemeMode.
Providers de dépendances	Injection des repositories et cas d'usage.

interventionsProvider est un AsyncNotifier. Lors d'un refresh réussi, il remplace les données. Lors d'un refresh échoué après un premier chargement valide, il conserve la valeur précédente et l'interface affiche un SnackBar. Si aucune valeur n'a encore été chargée, l'échec produit un état d'erreur avec l'action « Réessayer ».

Le provider familial du compte rendu isole l'état par identifiant d'intervention. Une écriture locale échouée conserve la dernière valeur valide. themeModeProvider expose la préférence globale et permet à MaterialApp de se reconstruire immédiatement.

7. Source JSON et Dio

7.1 Endpoint

```
GET https://utrera.ludovic.aflokkat-projet.fr/getInterventions.json
```

La source est distante, officielle et en lecture seule. L'application ne la modifie jamais et ne possède pas de backend local.

7.2 Configuration réseau

Paramètre	Valeur
Connexion	10 s
Envoi	10 s
Réception	15 s
Réponse attendue	JSON

7.3 Validation des données

- Code HTTP et nature objet de la racine JSON contrôlés par le RemoteDataSource.
- Champs requis imposés par les DTO Freezed.
- Chaînes métier non vides.
- Coordonnées finies et dans les bornes géographiques.
- Statuts/priorités connus et dates ISO valides.

Une donnée invalide devient un DataParsingFailure et n'entre pas dans le Domain.

7.4 Stratégie distant/local

```
JSON distant + surcharges Hive autorisées → données présentées
```

Le JSON reste la référence pour le technicien, la liste, la description, les coordonnées, les dates et l'historique. Le repository fusionne uniquement les statuts locaux par identifiant. Il n'existe pas de cache complet du JSON : un premier chargement et chaque actualisation nécessitent le réseau.

8. Persistance locale

Hive utilise exactement les quatre boxes suivantes :

Box	Clé	Valeur
intervention_statuses	Identifiant intervention	planifiée, en_cours ou terminée
intervention_notes	Identifiant intervention	Texte de la note
intervention_photos	Identifiant intervention	Chemin du fichier permanent
app_preferences	theme_mode	system, light ou dark

8.1 Statuts

Les statuts distants sont surchargés à la lecture par `intervention_statuses`. La transition métier est validée avant écriture : une intervention terminée ne peut plus progresser.

8.2 Notes

Une note est enregistrée indépendamment pour chaque intervention. Elle est relue à chaque ouverture du détail et persiste après fermeture et réouverture du stockage.

8.3 Photos

Principe important

Les photos ne sont jamais stockées dans Hive. Hive conserve uniquement le chemin de la copie permanente gérée par l'application.

7. `image_picker` sélectionne une image dans la galerie.
8. `path_provider` fournit le répertoire permanent de l'application.
9. Le fichier est copié dans le sous-répertoire `intervention_photos` avec un nom sûr et unique.
10. Hive enregistre uniquement le chemin de cette copie.
11. Après remplacement réussi, l'ancienne photo gérée est supprimée.
12. Si l'écriture Hive échoue, la nouvelle copie est nettoyée.

La suppression est volontairement limitée au répertoire géré afin de ne jamais supprimer le fichier source de l'utilisateur.

8.4 Préférence de thème

Le choix Système/Clair/Sombre est stocké sous `app_preferences/theme_mode`. La box est ouverte avant l'affichage de l'application et `themeModeProvider` restaure la valeur. `MaterialApp` observe ce provider et applique `theme`, `darkTheme` et `themeMode`.

9. Interface et responsive

9.1 Thèmes

`AppTheme` centralise les `ThemeData` Material 3 clair et sombre. Les écrans utilisent le `ColorScheme` au lieu de couleurs dispersées, ce qui maintient contraste, cohérence et identité visuelle.

9.2 Téléphone

- Navigation inférieure avec `NavigationBar`.
- Dashboard, Interventions et Profil comme destinations.
- Détail ouvert dans une page dédiée.
- Contenus verticaux et défilants.

9.3 Tablette

- À partir de 720 px : NavigationRail Material 3.
- Dashboard centré, contraint et réparti en groupes côte à côte.
- Profil centré avec largeur maximale.
- Master/détail : liste filtrable à gauche et détail à droite.
- Réutilisation du détail complet, y compris statut, historique, note et photo.

Validation responsive

Le breakpoint est centralisé dans ResponsiveBreakpoints. Les tests widgets couvrent notamment une surface simulée de 1200 × 900.

10. Gestion des erreurs

La hiérarchie AppFailure distingue les erreurs réseau, timeout, HTTP, données invalides, stockage local, transition de statut et erreur inconnue.

Contexte	Stratégie de reprise
Premier chargement échoué	État d'erreur explicite + bouton « Réessayer ».
Refresh échoué avec données valides	Conservation de la liste + SnackBar non destructif.
Écriture note/photo échouée	Conservation de la dernière valeur valide + message utilisateur.
Persistance du thème échouée	Retour au ThemeMode précédent.
Initialisation Hive impossible	Erreur explicite au bootstrap ; pas de mode dégradé silencieux.

Les exceptions techniques sont converties dans Data puis présentées sous forme de messages compréhensibles. Les widgets n'exposent pas d'exception brute à l'utilisateur.

11. Tests et validation

11.1 Suite automatisée

Résultat final

172 tests automatisés réussis. Aucun pourcentage de couverture n'est annoncé.

- Règles Domain et transitions de statut.
- Parsing, validation et mapping JSON ; erreurs réseau/HTTP/données invalides.
- Lecture, écriture, fermeture et réouverture de Hive.
- Fusion des statuts distants et locaux ; repositories et cas d'usage.
- Providers Riverpod, recherche, filtres, tris, dashboard et pull-to-refresh.
- Détail, historique, progression du statut, note et photo.
- Préférence et application du thème.
- Navigation téléphone/tablette, master/detail, dashboard et profil à 1200 × 900.
- Absence d'overflow dans les tailles ciblées.

Les tests photo utilisent des abstractions injectables, des boxes temporaires et des répertoires temporaires : aucun accès réel à la photothèque de l'utilisateur n'est requis.

11.2 Validation Android

- flutter build apk --debug réussi.
- Samsung SM S926U sous Android 16/API 36 détecté.
- APK installé et application lancée ; démarrage Flutter/Impeller observé sans erreur applicative visible.
- Parcours tactile complet de la photothèque non revendiqué.
- Persistance du thème après fermeture/reprise non revendiquée comme test manuel.

11.3 Validation tablette

Les comportements tablette sont vérifiés par tests widgets à 1200 × 900. Aucune tablette physique et aucun émulateur tablette n'ont été utilisés.

11.4 Validation iOS

La cible iOS est générée et la description d'accès à la photothèque est configurée. Elle n'a pas été compilée dans l'environnement Windows ; une validation iOS nécessite macOS et Xcode.

12. Difficultés rencontrées et solutions

Difficulté	Solution mise en œuvre
Conserver le JSON comme référence tout en modifiant un statut	Fusion ciblée des statuts Hive par identifiant dans le repository.
Actualiser sans effacer une liste utilisable	Conservation de l'AsyncData précédent et notification par Snackbar.
Garantir un tri déterministe	Comparaison métier puis conservation de l'ordre initial en cas d'égalité.
Pérenniser une photo choisie dans la galerie	Copie dans le stockage applicatif avant enregistrement du chemin.
Éviter les fichiers orphelins lors d'un remplacement	Ordre transactionnel simple et nettoyage compensatoire.
Restaurer le thème avant l'affichage	Ouverture de app_preferences au bootstrap et lecture par le provider.
Partager les fonctions mobile et tablette	Shell responsive et détail réutilisable en page ou panneau intégré.
Tester les photos sans stockage utilisateur	Abstractions injectables et répertoires temporaires.

13. Limites connues

- Connexion réseau indispensable au premier chargement et aux actualisations.
- Aucun cache complet offline-first du JSON.
- Aucune synchronisation serveur des statuts, notes ou photos.
- Aucune authentification réelle ni résolution de conflit multi-appareil.
- Historique distant non enrichi par les actions locales.
- Sélection de photo depuis la galerie uniquement.
- Pas de récupération spécifique d'une sélection interrompue par destruction du processus Android.
- Parcours tactile photo complet non validé dans cette livraison.
- Persistance du thème non vérifiée manuellement après fermeture/reprise.
- Tablette validée par tests widgets uniquement.
- Cible iOS non compilée sous Windows.

14. Améliorations futures

- Authentification et gestion des habilitations.
- API d'écriture et synchronisation serveur.
- Cache JSON complet et stratégie offline-first.
- Résolution des conflits et journal de synchronisation.
- Cartographie des stations.
- Internationalisation.
- Ajout local d'événements dans l'historique.
- Compression et téléversement des photos.
- Récupération Android des sélections perdues.
- Animations et transitions avancées.
- Validation sur tablette physique et chaîne CI macOS/iOS.

15. Commandes de reproduction

```
flutter pub get  
dart format .  
flutter analyze  
flutter test  
flutter build apk --debug
```

Après modification des DTO générés :

```
dart run build_runner build
```

16. Conclusion

La livraison finale couvre le socle fonctionnel et les quatre extensions optionnelles. L'architecture sépare les règles métier des détails Flutter, réseau et stockage. Les données distantes demeurent la référence, tandis que Hive et le stockage applicatif prennent en charge les informations locales. Les validations automatisées couvrent les scénarios fonctionnels, les erreurs et les deux familles de layouts, dans les limites de plateformes déclarées ci-dessus.

Bilan

AgriVista Field est livré avec 172 tests automatisés réussis, un build Android valide et une documentation alignée sur le comportement réellement implémenté.

Annexe A — Matrice de conformité au sujet

A.1 Socle obligatoire et exigences de remise

Exigence	État	Preuve technique
JSON distant officiel	Conforme	Dio et endpoint centralisé.
DTO et parsing typé	Conforme	Freezed, json_serializable, mapper validant.
Liste des interventions	Conforme	Page, cartes et états asynchrones.
Recherche	Conforme	Station, domaine et description.
Filtres statut/priorité	Conforme	3 statuts + 3 priorités.
Détail et date prévue	Conforme	Page/panneau de détail.
Historique distant	Conforme	Affichage en lecture seule.
Progression du statut	Conforme	Cas d'usage Domain séquentiel.
Persistance du statut	Conforme	intervention_statuses.
Erreur Hive explicite	Conforme	LocalStorageFailure et vue d'erreur.
Profil dynamique	Conforme	Technicien issu du provider global.
Navigation commune	Conforme	Shell Material 3.
Domain en Dart pur	Conforme	Aucun import technique dans domain/.
Dossier de conception	Conforme	PDF v1.1 présent dans docs/ et aligné sur les quatre extensions.
Dépôt Git privé	Configuré	Remote origin : DorianArg/AgriVistaEXAM ; accès pédagogique à vérifier au moment de la remise.
Flutter Android	Validé techniquement	APK debug construit ; application installée et lancée sur Samsung SM S926U, Android 16/API 36.
Flutter iOS	À valider sur macOS	Projet iOS généré et accès photothèque configuré ; aucune compilation iOS sous Windows.
Dossier technique	Conforme	DOCX/PDF final + source docs/dossier_technique.md.

A.2 Extensions optionnelles

Extension	État	Preuve technique
Dashboard	Réalisée	dashboardSummaryProvider et cartes filtrantes.
Pull-to-refresh	Réalisée	RefreshIndicator et recharger().
Tri date/priorité/statut	Réalisé	InterventionSort + direction réversible.
Note locale	Réalisée	intervention_notes + formulaire terrain.
Photo locale	Réalisée	image_picker, copie permanente, chemin Hive.

Extension	État	Preuve technique
Système/Clair/Sombre	Réalisés	ThemeMode, AppTheme et app_preferences.
Responsive tablette	Réalisé	Breakpoint 720 px et NavigationRail.
Master/detail	Réalisé	Liste et détail simultanés sur tablette.

Annexe B — Livraison et versionnement

Le projet a été développé par étapes avec des commits significatifs : initialisation, Domain et chargement distant, persistance Hive, liste/recherche/filtres, détail et statut, profil, dashboard, pull-to-refresh et tri, compte rendu terrain (note/photo), thème responsive tablette, puis documentation finale.

Le dépôt distant configuré est :

<https://github.com/DorianArg/AgriVistaEXAM>

Le tag v1.0-socle conserve le point de référence correspondant au périmètre obligatoire validé avant l'ajout des fonctionnalités optionnelles.

Les éléments locaux suivants restent exclus du versionnement :

- android/local.properties ;
- .dart_tool/ ;
- build/ et les APK générés ;
- .idea/ ;
- caches, fichiers temporaires Flutter/IDE et autres artefacts locaux.

Livrables de remise :

- dépôt Git privé AgriVistaEXAM ;
- dossier de conception v1.1 au format PDF ;
- dossier technique final au format PDF ;
- README.md et source Markdown docs/dossier_technique.md ;
- sources Flutter Android/iOS et tests automatisés.

Avant l'envoi définitif, il reste uniquement à vérifier l'accès de l'équipe pédagogique au dépôt privé et la présence des fichiers finaux dans docs/. La cible iOS reste explicitement à compiler sur macOS/Xcode ; cette limite est documentée et ne doit pas être présentée comme une validation réalisée sous Windows.